CENTRO UNIVERSITÁRIO UNIFECAF
ENGENHARIA DA COMPUTAÇÃO

Disciplina: Sistemas Operacionais

KernelLab: simulação de escalonamento, memória virtual e controle de dispositivos no cenário MedControl

Relatório técnico — Parte teórica

Kaio Santana Rodrigues Santos
Matrícula 98328

2026

Sumário

1. Contextualização do problema
2. Estados e ciclo de vida dos processos
3. Algoritmos de escalonamento
4. Conceitos e métricas de desempenho
5. Resultados e comparação dos escalonadores
6. Memória virtual
7. Algoritmos de substituição: FIFO, LRU e Optimal
8. Concorrência, mutex e semáforos
9. Segurança no acesso aos dispositivos
10. Relação entre a teoria e as decisões implementadas
11. Recomendação técnica e limitações
Referências

Repositório do simulador: https://github.com/kaiooo20/kernellab

1. Contextualização do problema

A MedControl opera um sistema de monitoramento hospitalar em que tarefas de naturezas muito diferentes disputam os mesmos recursos de hardware. Na mesma CPU executam o monitoramento cardíaco do paciente (P1), a transmissão de telemetria (P2), a geração de relatórios (P3), a emissão de alarmes críticos (P4), o backup (P5) e a atualização de dados (P6). Esses processos também concorrem por memória (de 60 MB a 300 MB cada) e por dispositivos compartilhados: disco, rede, sistema de áudio e arquivos de log.

O sistema operacional é o componente que decide quem usa cada recurso e em que ordem. Em um computador de escritório, uma decisão ruim custa alguns segundos de lentidão; no cenário da MedControl, a mesma decisão pode atrasar o alarme sonoro de uma parada cardíaca porque o processador está ocupado com um backup. Por isso o problema não é escolher o algoritmo "mais rápido" em média, mas o que equilibra **desempenho**, **respeito à prioridade**, **justiça** entre os processos e **criticidade** das tarefas de saúde.

Para fundamentar essa escolha foi desenvolvido o **KernelLab**, um simulador em Python que reproduz as decisões de um SO — escalonamento da CPU, substituição de páginas e controle de dispositivos — com os dados reais do cenário e mede o efeito de cada política. Este relatório apresenta a teoria que sustenta o simulador, sempre ligada aos números que ele produziu.

Processo	Função	Chegada	Burst	Prioridade	Memória	Dispositivo
P1	Monitoramento cardíaco	0	8	1	120 MB	Rede
P2	Transmissão de telemetria	1	4	2	80 MB	Rede
P3	Geração de relatórios	2	9	4	200 MB	Disco
P4	Emissão de alarme crítico	3	2	0	60 MB	Áudio e rede
P5	Backup do sistema	5	12	5	300 MB	Disco
P6	Atualização de dados	6	6	3	150 MB	Disco e rede

Tabela 1 — Processos do cenário (menor número = maior prioridade).

Uma leitura inicial da tabela já revela a tensão central do caso: o processo mais crítico (P4, prioridade 0) é também o que chega depois de três outros e tem o menor burst; os processos administrativos P3 e P5 são longos e de baixa prioridade. Qualquer política que ignore a prioridade colocará o alarme atrás de tarefas longas; qualquer política que só olhe a prioridade arrisca deixar backup e relatórios esperando indefinidamente.

2. Estados e ciclo de vida dos processos

Um processo é um programa em execução, com seu próprio contexto: registradores, contador de programa, espaço de endereçamento e recursos abertos. O SO mantém essas informações no Bloco de Controle de Processo (PCB) e, ao longo da vida, o processo passa pelos estados abaixo (Tanenbaum e Bos, cap. 2).

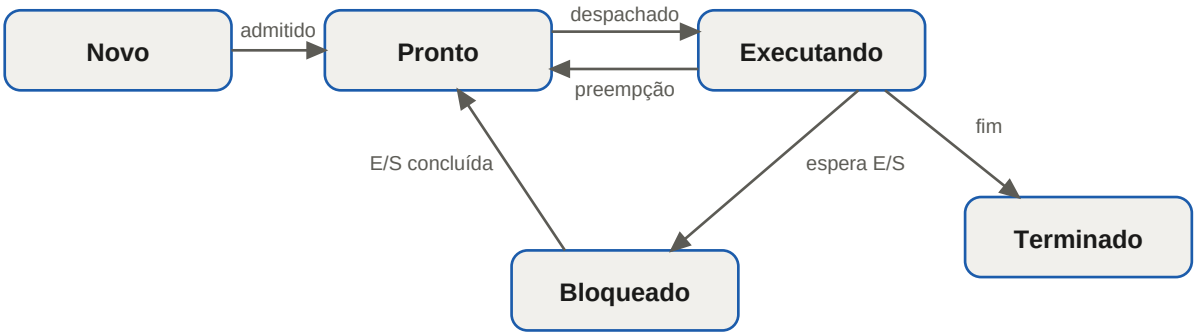


Figura 1 — Diagrama de estados de um processo.

- **Novo**: o processo está sendo criado; o SO aloca o PCB e verifica se há memória. No cenário, P5 exige 300 MB; se não houver memória livre, ele não pode ser admitido.
- **Pronto**: tem tudo para executar, só falta a CPU. É nessa fila que o escalonador atua; o tempo de espera medido pelo simulador é exatamente o tempo passado aqui.
- **Executando**: ocupa a CPU. Com uma única CPU, só um processo por vez está neste estado — é o que cada barra do Diagrama de Gantt representa.
- **Bloqueado**: aguarda um evento, como o fim de uma gravação em disco ou o acesso a um dispositivo ocupado. Na Etapa 4, P6 fica bloqueado 6 unidades esperando o disco que o backup usa em modo exclusivo.
- **Terminado**: concluiu; o SO libera memória e recursos. O instante de término define o tempo de retorno.

As transições Executando → Pronto (preempção) e Pronto → Executando (despacho) são as **trocas de contexto**: o SO salva o estado do processo que sai no PCB e restaura o do que entra. Elas são custo puro — nenhum trabalho útil é feito durante a troca — e por isso o simulador as contabiliza. No cenário da MedControl, a transição mais importante é a preempção: é ela que permite tirar o monitoramento cardíaco da CPU no instante 3 para dar lugar ao alarme.

3. Algoritmos de escalonamento

O escalonador de curto prazo escolhe, entre os processos prontos, qual recebe a CPU. Algoritmos **não preemptivos** só trocam de processo quando o atual termina ou bloqueia; **preemptivos** podem interromper o processo em execução — por tempo (quantum) ou pela chegada de alguém mais importante. Foram implementados os quatro algoritmos exigidos e duas variantes de prioridade para isolar o efeito da preempção e do aging.

3.1 FCFS — First Come, First Served

Atende pela ordem de chegada, como uma fila de banco, sem preempção. É simples, tem custo de decisão mínimo e nunca causa starvation: todo processo acaba sendo atendido. O problema é o **efeito comboio**: processos curtos ficam presos atrás de longos. Na simulação, o alarme P4 chegou no instante 3 mas só executou no instante 21, depois de P1, P2 e P3 — 18 unidades de espera para uma tarefa de 2. Para a MedControl, FCFS é inaceitável justamente porque trata o alarme como

qualquer outra tarefa.

3.2 SJF — Shortest Job First

Escolhe o processo pronto com o menor burst. É comprovadamente ótimo para o tempo médio de espera entre os não preemptivos, porque executar primeiro os curtos reduz a espera acumulada de todos. Tem duas limitações: exige conhecer o burst de antemão (um SO real precisa estimá-lo, por exemplo por média exponencial dos bursts anteriores) e favorece sistematicamente os curtos, o que pode causar **starvation** dos longos. Na simulação, P5 (backup, 12 unidades) foi o último a executar e esperou 24 unidades. O alarme foi beneficiado só por coincidência — por ser curto, não por ser crítico.

3.3 Round Robin (quantum = 3)

Cada processo usa a CPU por no máximo um quantum e volta ao fim da fila. É o algoritmo clássico de tempo compartilhado: nenhum processo espera mais que (n − 1) × quantum para voltar a executar, o que garante bom tempo de resposta e justiça. O preço são as trocas de contexto e o aumento do tempo de retorno, porque todos os processos avançam "um pouco de cada vez". A escolha do quantum é decisiva: muito grande, o RR vira FCFS (o simulador comprova isso com quantum 100); muito pequeno, o sistema gasta mais tempo trocando de contexto do que trabalhando. Com quantum 3 foram 13 trocas de contexto, mais que o dobro dos outros algoritmos, e o alarme ainda esperou 6 unidades na fila circular.

3.4 Escalonamento por prioridade

Cada processo tem uma prioridade e a CPU vai para o de maior prioridade (menor número). Na versão **preemptiva**, adotada como principal, a chegada de um processo mais prioritário interrompe o atual. É a única política que traduz diretamente a importância clínica das tarefas em decisão de escalonamento: no instante 3, P4 (prioridade 0) interrompeu P1 e executou imediatamente. O risco é a starvation: um fluxo constante de processos prioritários pode adiar indefinidamente os de baixa prioridade. A solução clássica é o **aging** — aumentar gradualmente a prioridade de quem espera. A variante implementada melhora a prioridade em 1 nível a cada 4 unidades de espera, com um piso: nenhum processo comum alcança o nível 0, reservado ao alarme.

4. Conceitos e métricas de desempenho

Para comparar os algoritmos de forma objetiva, o simulador calcula, para cada processo:

- **Tempo de retorno** (turnaround) = término − chegada: quanto tempo o processo ficou no sistema. Importa para tarefas em lote como relatórios e backup.
- **Tempo de espera** = retorno − burst: tempo total na fila de prontos. É o que o escalonador controla diretamente.
- **Tempo de resposta** = primeira execução − chegada: quanto tempo até o processo começar a ser atendido. É a métrica mais importante para o alarme e para o monitoramento.
- **Retorno normalizado** = retorno ÷ burst: quantas vezes o tempo de permanência excede o trabalho útil. Permite comparar processos de tamanhos diferentes; o alarme no FCFS chega a

10, ou seja, ficou 10 vezes mais tempo no sistema do que precisava.

E, para cada algoritmo: médias das três métricas, número de **trocas de contexto**, utilização da CPU e o **índice de justiça de Jain**, calculado sobre a fração do tempo de permanência em que cada processo efetivamente usou a CPU (burst ÷ retorno). O índice vale 1 quando todos são tratados de forma igual e cai à medida que alguns processos são privilegiados à custa de outros.

O simulador também sinaliza dois tipos de risco. **Atraso excessivo** ocorre quando a espera supera 1,5 vez a média ou o retorno normalizado atinge 4. **Risco de starvation** é marcado quando o processo foi ultrapassado por processos que chegaram depois dele e ainda esperou pelo menos a média: numa simulação finita todos terminam, mas esse padrão indica que, sob chegada contínua de processos favorecidos, ele poderia esperar indefinidamente. Os limites são configuráveis em *config.json*.

5. Resultados e comparação dos escalonadores

A Figura 2 mostra os Diagramas de Gantt produzidos pelo simulador e a Tabela 2 resume as métricas.

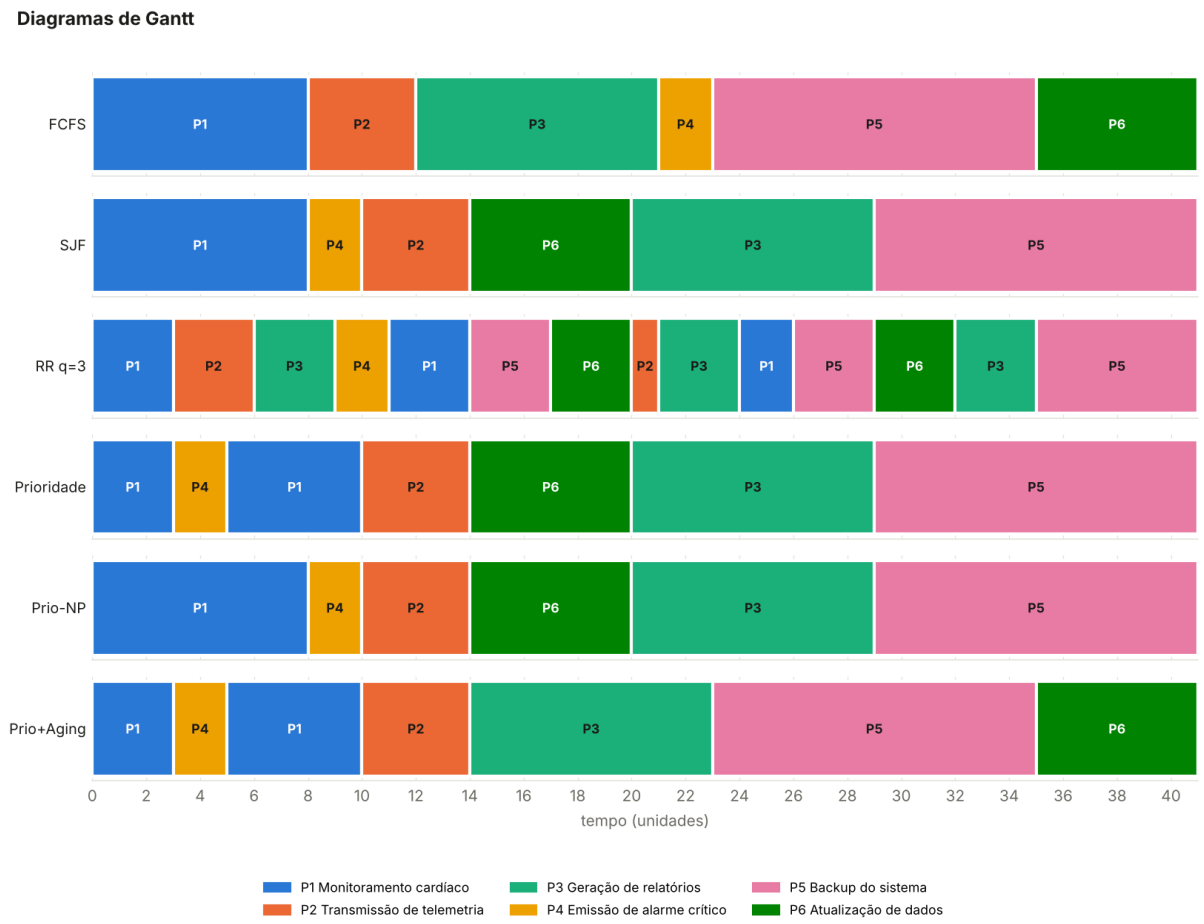


Figura 2 — Diagramas de Gantt gerados pelo KernelLab (quantum 3; aging a cada 4 unidades).

Algoritmo	Espera méd.	Retorno méd.	Resposta méd.	Trocas	Resp./Ret. P4	Jain	Starvation
FCFS	13,67	20,50	13,67	5	18 / 20	0,674	—
SJF	10,67	17,50	10,67	5	5 / 7	0,761	P3, P5
RR q=3	18,00	24,83	5,33	13	6 / 8	0,972	—
Prioridade	10,17	17,00	9,83	6	0 / 2	0,799	P3, P5
Prio-NP	10,67	17,50	10,67	5	5 / 7	0,761	P3, P5
Prio+Aging	11,67	18,50	11,33	6	0 / 2	0,764	P3

Tabela 2 — Métricas por algoritmo. Prio-NP = prioridade não preemptiva.

Menor tempo médio de espera: prioridade preemptiva (10,17), seguida de perto por SJF e prioridade não preemptiva (10,67). Nesta carga, a ordem por prioridade coincide em boa parte com a ordem por tamanho, e a preempção do alarme ainda reduz a espera dele a zero.

Melhor tempo de resposta: Round Robin (5,33). Como todos recebem a CPU logo na primeira volta da fila, ninguém espera muito para começar — mas "começar" não é "terminar": o RR teve o pior tempo médio de retorno (24,83).

Alarme tratado com maior eficiência: prioridade preemptiva (com ou sem aging): resposta 0 e retorno 2, isto é, o alarme executou no instante em que chegou. No FCFS a resposta foi 18; no SJF e na prioridade não preemptiva, 5, porque o alarme teve de esperar o monitoramento cardíaco terminar; no RR, 6.

Maior risco de starvation: SJF e prioridade (preemptiva e não preemptiva), empatados. Em todos eles P3 e P5 são ultrapassados por processos que chegaram depois e esperam 18 e 24 unidades. No SJF o critério é o tamanho e na prioridade é a importância, mas o efeito é o mesmo. FCFS e RR não apresentam o risco por construção.

Distribuição mais justa: Round Robin (Jain 0,972), muito acima dos demais (0,67 a 0,80). Todos os processos receberam aproximadamente a mesma fração de CPU durante sua permanência.

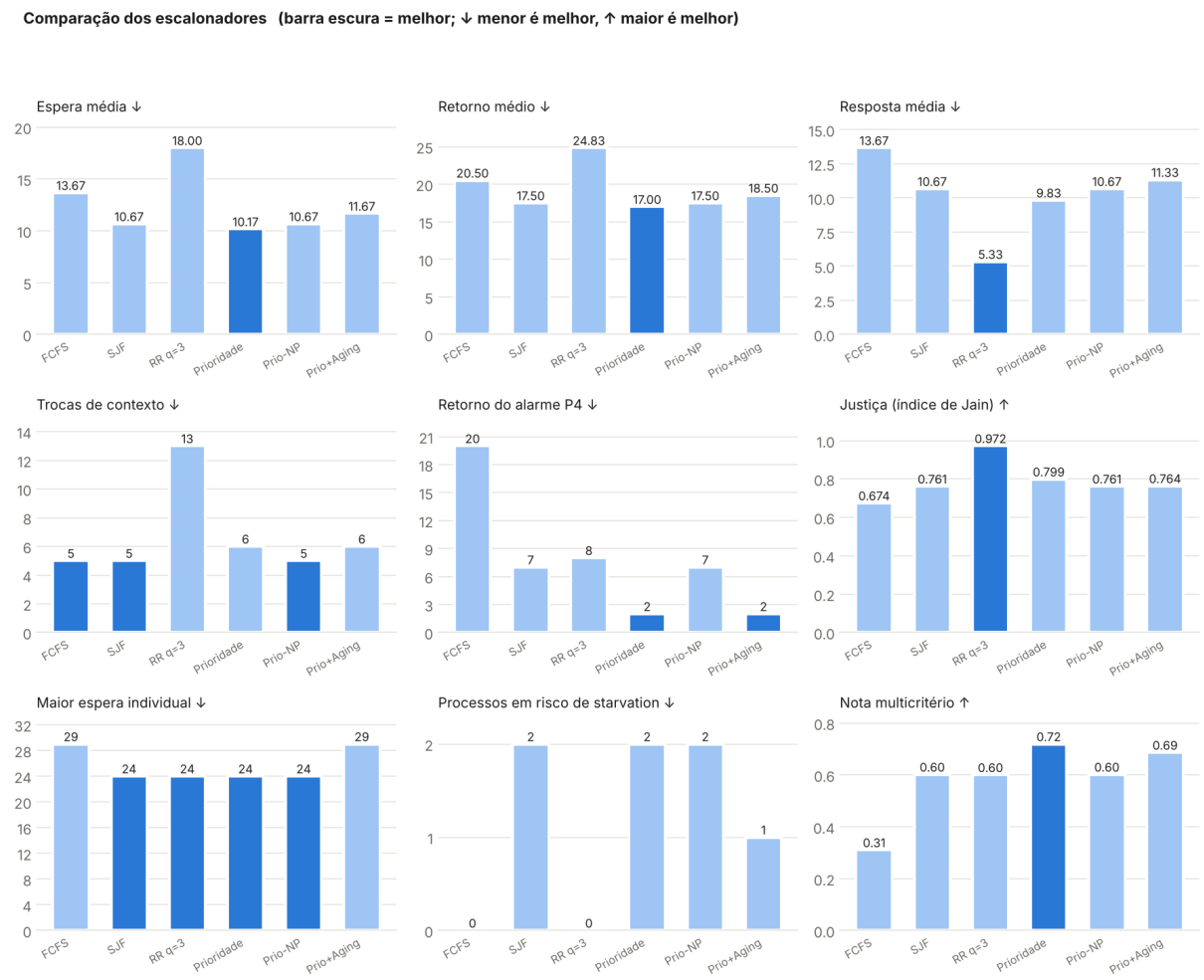


Figura 3 — Comparação dos escalonadores (barra escura indica o melhor em cada critério).

Mais adequado para a MedControl: nenhum algoritmo vence em todos os critérios, então a escolha precisa de pesos explícitos. O simulador calcula uma nota multicritério normalizando cada critério entre 0 e 1 e aplicando pesos: tratamento do alarme 30%, espera média 20%, resposta média 15%, justiça 15%, starvation 10% e trocas de contexto 10%. O alarme recebe o maior peso porque, num hospital, seu atraso tem consequência clínica direta. A prioridade preemptiva obteve a maior nota (0,72), seguida pela variante com aging (0,69); SJF e RR empataram em 0,60 e FCFS ficou com 0,31.

A variante com aging merece leitura cuidadosa. Ela reduziu a espera do backup (P5) de 24 para 18 e o número de processos em risco de starvation de 2 para 1, sem afetar o alarme. Em troca, P6 passou a ser o último e a espera média subiu de 10,17 para 11,67. Numa carga curta e finita como esta, o aging apenas redistribui a espera; seu valor aparece em operação contínua, quando novos alarmes e pacotes de telemetria chegam o tempo todo e, sem ele, o backup poderia nunca executar.

6. Memória virtual

A memória virtual dá a cada processo a ilusão de um espaço de endereçamento próprio e contíguo, maior que a memória física disponível. O espaço lógico é dividido em **páginas** de tamanho fixo e a memória física em **frames** (molduras) do mesmo tamanho. A unidade de gerenciamento de

memória (MMU) traduz cada endereço virtual em físico consultando a tabela de páginas do processo, com a TLB funcionando como cache das traduções mais recentes.

Só as páginas em uso precisam estar na memória (paginação sob demanda). Quando o processo referencia uma página ausente, ocorre uma **falta de página** (page fault): o processo é bloqueado, o SO busca a página no disco e, se não houver frame livre, escolhe uma página para remover — a decisão do algoritmo de substituição. Como um acesso a disco é milhares de vezes mais lento que um acesso à RAM, a taxa de faltas domina o desempenho. Uma taxa alta leva ao **thrashing**, situação em que o sistema passa mais tempo trocando páginas do que executando.

No cenário da MedControl, os seis processos somam 910 MB. Se a memória física for menor, nem todos cabem ao mesmo tempo, e cada falta de página do monitoramento cardíaco significa um processo crítico bloqueado esperando o disco — o mesmo disco disputado pelo backup. Um bom algoritmo de substituição, portanto, protege indiretamente as tarefas críticas.

7. Algoritmos de substituição: FIFO, LRU e Optimal

Os três algoritmos foram simulados com 4 frames e a sequência de referências 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 2, 1, 5, 3, 2 (17 referências).

- **FIFO** remove a página carregada há mais tempo. É simples (uma fila basta), mas ignora o uso: uma página muito acessada é removida só por ser antiga. Também sofre da **anomalia de Belady** — aumentar os frames pode aumentar as faltas, como os testes do projeto demonstram com a sequência clássica (9 faltas com 3 frames e 10 com 4).
- **LRU** (Least Recently Used) remove a página usada há mais tempo. Baseia-se na **localidade temporal**: o que foi usado recentemente tende a ser usado de novo. É imune à anomalia de Belady, mas exige registrar cada acesso; na prática, os SOs usam aproximações como o algoritmo do relógio (segunda chance).
- **Optimal** (OPT, de Belady) remove a página que demorará mais para ser usada de novo. Produz o menor número possível de faltas, mas exige conhecer o futuro, por isso não é implementável — serve como referência para medir os outros.

Algoritmo	Page faults	Page hits	Taxa de faltas	Distância do ótimo
FIFO	12	5	70,59%	5
LRU	10	7	58,82%	3
Optimal	7	10	41,18%	0

Tabela 3 — Resultados da substituição de páginas (4 frames).

Estado dos frames — FIFO

Ref.	1	2	3	4	1	2	5	1	2	3	4	5	2	1	5	3	2
F0	1	1	1	1	1	1	5	5	5	5	4	4	4	4	4	4	4
F1		2	2	2	2	2	2	1	1	1	1	5	5	5	5	5	5
F2			3	3	3	3	3	3	2	2	2	2	2	1	1	1	1
F3				4	4	4	4	4	4	3	3	3	3	3	3	3	2
	F	F	F	F	·	·	F	F	F	F	F	F	·	F	·	·	F

F = falta de página; · = acerto. 12 faltas e 5 acertos.

Estado dos frames — LRU

Ref.	1	2	3	4	1	2	5	1	2	3	4	5	2	1	5	3	2
F0	1	1	1	1	1	1	1	1	1	1	1	5	5	5	5	5	5
F1		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
F2			3	3	3	3	5	5	5	5	4	4	4	4	4	3	3
F3				4	4	4	4	4	4	3	3	3	3	1	1	1	1
	F	F	F	F	·	·	F	·	·	F	F	F	·	F	·	F	·

F = falta de página; · = acerto. 10 faltas e 7 acertos.

Estado dos frames — Optimal

Ref.	1	2	3	4	1	2	5	1	2	3	4	5	2	1	5	3	2
F0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3	3
F1		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
F2			3	3	3	3	3	3	3	3	4	4	4	4	4	4	4
F3				4	4	4	5	5	5	5	5	5	5	5	5	5	5
	F	F	F	F	·	·	F	·	·	·	F	·	·	·	·	F	·

F = falta de página; · = acerto. 7 faltas e 10 acertos.

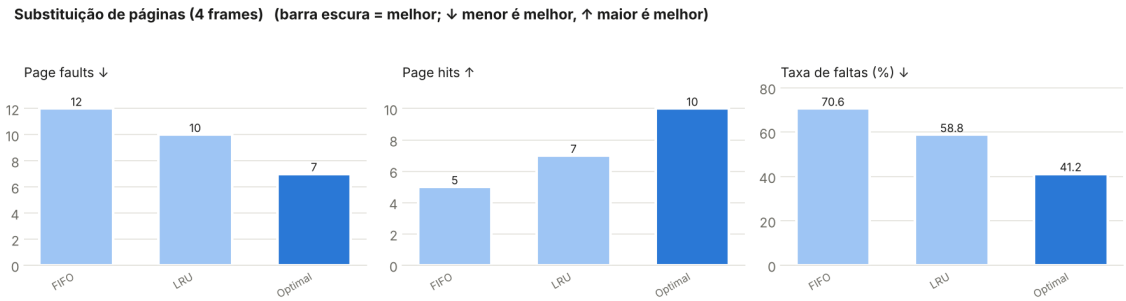


Figura 4 — Page faults, hits e taxa de faltas.

Justificativa da diferença. O ponto decisivo é a referência 7 (página 5). O FIFO remove a página 1, a mais antiga — que é usada logo em seguida. Isso inicia uma cascata: para trazer a 1 de volta ele remove a 2, para trazer a 2 remove a 3, e assim por diante; o FIFO erra seis vezes seguidas (referências 7 a 12). O LRU, no mesmo ponto, remove a página 3, a menos usada recentemente, e mantém 1 e 2, que são exatamente as próximas referências. O Optimal remove a 4, que é a página mais distante no futuro. O LRU ficou a 3 faltas do ótimo e o FIFO a 5. Como a sequência tem forte localidade temporal (as páginas 1, 2 e 5 voltam repetidamente), o algoritmo que olha o uso recente vence o que olha apenas a idade. Para a MedControl, isso significa preferir LRU ou suas

aproximações: o laço do monitoramento cardíaco acessa sempre as mesmas páginas e não deve perdê-las para páginas do backup só por terem sido carregadas antes.

8. Concorrência, mutex e semáforos

Processos e threads concorrentes compartilham recursos. Quando o resultado depende da ordem em que as operações se intercalam, há uma **condição de corrida**. O trecho de código que acessa o recurso compartilhado é a **seção crítica**, e uma solução correta precisa garantir exclusão mútua (um por vez), progresso (ninguém fica bloqueado sem necessidade) e espera limitada.

Um **mutex** é um trinco com dois estados: quem o obtém entra na seção crítica e os demais esperam até a liberação. Um **semáforo**, proposto por Dijkstra, generaliza a ideia com um contador inteiro e duas operações atômicas: P (wait/down) decrementa e bloqueia se não houver vaga; V (signal/up) incrementa e acorda um processo em espera. Um semáforo iniciado em 1 é um mutex; iniciado em N, permite até N usuários simultâneos.

O KernelLab demonstra os dois de duas formas:

- **Simulação determinística dos dispositivos**: cada dispositivo tem uma fila FIFO e um semáforo com a sua capacidade — disco, áudio e logs com capacidade 1 (mutex) e rede com 2. Pedidos exclusivos só são atendidos com o dispositivo vazio e ocupam todas as vagas. A cada instante o simulador verifica as invariantes e não registrou nenhuma violação. No disco, P3 grava de 0 a 3, o backup P5 pede acesso exclusivo no instante 1 e só entra no instante 3, e P6 espera até o instante 7.
- **Threads reais do Python**: quatro threads incrementam o contador de linhas do log 200 vezes cada. Sem lock, a sequência ler → trocar de thread → gravar perde atualizações (na execução registrada, o contador terminou em 202 em vez de 800). Com *threading.Lock*, o resultado é sempre 800.

No contexto hospitalar, a condição de corrida não é abstrata: duas escritas intercaladas no log de auditoria geram registros corrompidos, e duas gravações simultâneas no mesmo prontuário podem misturar dados de pacientes. O uso descuidado de mutexes, porém, traz outros riscos: **deadlock** (P6 segurando a rede e esperando o disco enquanto outro processo faz o inverso) e **inversão de prioridade** (o alarme esperando um recurso preso por um processo de baixa prioridade). Esses riscos são discutidos como limitação na seção 11.

9. Segurança no acesso aos dispositivos

Proteção é o mecanismo que controla quais processos podem acessar quais recursos e de que forma. O modelo conceitual é a **matriz de controle de acesso**: linhas são sujeitos (processos), colunas são objetos (dispositivos) e cada célula contém os direitos. Na prática, o SO a armazena por coluna (listas de controle de acesso, ACLs) ou por linha (capabilities). A matriz do KernelLab foi construída pelo **princípio do menor privilégio**: cada processo recebe apenas o que sua função exige.

Processo	Disco	Rede	Áudio	Logs
P1	———	RW–U–	———	–W–U–
P2	———	RW–U–	———	–W–U–
P3	RW–U–	———	———	R—U–
P4	———	–W–U–	–WXUE	–W–U–
P5	RW–UE	———	———	RW–U–
P6	RW–U–	R—U–	———	–W–U–

Tabela 4 — Matriz de controle de acesso (R ler, W gravar, X executar, U utilizar, E solicitar acesso exclusivo).

- Só o alarme (P4) pode **executar** rotinas e pedir acesso exclusivo ao **áudio**: um processo comprometido de telemetria não pode disparar nem silenciar alarmes. O simulador nega o pedido de P2 ao áudio.
- Só o backup (P5) pode pedir o **disco** com exclusividade, garantindo uma cópia consistente. O pedido exclusivo de P6 é negado.
- O **log** de auditoria é append-only para quase todos: os processos gravam, mas só relatórios (P3) e backup (P5) podem lê-lo, e P3 não pode gravar nele. Isso preserva a rastreabilidade exigida em sistemas de saúde.
- P3 não tem acesso à **rede**: relatórios com dados de pacientes não saem do equipamento por esse processo. O pedido de P3 à rede é negado antes de entrar na fila.

A verificação acontece antes do enfileiramento: um pedido não autorizado nunca chega a disputar o recurso. Separar "quem pode" (matriz) de "quando pode" (semáforo) é a mesma divisão que um SO real faz entre o subsistema de proteção e o de sincronização.

10. Relação entre a teoria e as decisões implementadas

Conceito	Decisão no KernelLab	Consequência observada
Preempção	Prioridade preemptiva simulada unidade a unidade	Alarme executa no instante em que chega (resposta 0)
Troca de contexto	Contada a cada mudança entre processos diferentes	Revela o custo do RR (13 trocas contra 5 ou 6)
Starvation e aging	Detecção por ultrapassagem + variante com aging limitado	Backup espera 18 em vez de 24; aging nunca alcança o nível do alarme
Justiça	Índice de Jain sobre burst/retorno	Quantifica que o RR é o mais justo (0,972)
Localidade temporal	LRU com registro do último acesso	10 faltas contra 12 do FIFO
Limite teórico	Optimal como referência, não como opção	Mostra que o LRU está a 3 faltas do mínimo possível
Exclusão mútua	Semáforo contador por dispositivo + checagem de invariantes	Nenhum acesso simultâneo indevido registrado
Menor privilégio	Matriz de acesso verificada antes da fila	3 pedidos indevidos negados
Separação política × mecanismo	Dados em JSON e algoritmos independentes	Processos, quantum, frames e páginas alteráveis sem mudar o código

Tabela 5 — Conceitos teóricos e como cada um aparece no simulador.

A última linha resume a arquitetura. Os escalonadores apenas decidem quem usa a CPU e quando (devolvem a linha do tempo); as métricas são calculadas depois, por um módulo independente. Esse é o princípio de separação entre política e mecanismo e foi o que permitiu comparar seis políticas com o mesmo código de medição. A suíte de 32 testes automatizados confere os resultados contra valores calculados à mão.

11. Recomendação técnica e limitações

Para o cenário da MedControl recomenda-se o **escalonamento por prioridade preemptivo com aging**, organizado em classes: o alarme crítico no nível mais alto, sempre capaz de interromper qualquer processo; monitoramento e telemetria em seguida; tarefas administrativas (atualização, relatórios e backup) por último, protegidas da starvation pelo aging, que tem um piso para nunca as igualar ao alarme. Dentro de uma mesma classe, o Round Robin pode distribuir a CPU de forma justa, o que resulta numa estrutura de filas multinível. Essa recomendação vem da simulação: a prioridade preemptiva foi a única que atendeu o alarme sem nenhuma espera, teve a menor espera média e a maior nota multicritério.

Na memória, recomenda-se LRU ou uma aproximação eficiente (relógio), dado o ganho de 2 faltas sobre o FIFO e a imunidade à anomalia de Belady. Nos dispositivos, a matriz de acesso pelo menor privilégio e mutex ou semáforo por dispositivo.

Limitações que permanecem:

- Modelo de CPU única, sem custo explícito de troca de contexto e sem bloqueio por E/S durante o escalonamento.
- Bursts conhecidos de antemão; um SO real precisaria estimá-los, o que reduz a vantagem do SJF.
- Memória e dispositivos simulados de forma independente da CPU: uma falta de página não bloqueia o processo no Gantt.
- Fila FIFO nos dispositivos: o alarme pode esperar a rede ocupada (esperou 1 unidade na simulação). Uma melhoria é usar fila por prioridade com herança de prioridade, que evita a inversão de prioridade, e detecção ou prevenção de deadlock.
- Uma única carga de trabalho e tempo discreto: os resultados valem para este cenário. Por isso o simulador aceita outras cargas e parâmetros pela linha de comando.

Referências

TANENBAUM, Andrew S.; BOS, Herbert. *Sistemas operacionais modernos*. 4. ed. São Paulo: Pearson, 2016. Disponível em: https://www.amazon.com.br/Sistemas-operacionais-modernos-Andrew-Tanenbaum-ebook/dp/B01IPIN4W4.

SANTANA, Max. *Organização e Arquitetura de Computadores II: sistemas multiprogramados*. Univasf. Disponível em: https://www.docentes.univasf.edu.br/max.santana/material/OAC-II/sistemas_multiprogramados.pdf.

ENDLER, Markus. *INF 1316: Sistemas Operacionais*. PUC-Rio. Disponível em: https://www-di.inf.puc-rio.br/~endler/courses/inf1019/transp/aulas-teoricas/cap-1.pdf.

TECNOBLOG. *O que é um sistema operacional?* Disponível em: https://tecnoblog.net/responde/o-que-e-um-sistema-operacional/.

JAIN, Raj; CHIU, Dah-Ming; HAWE, William. *A quantitative measure of fairness and discrimination for resource allocation in shared computer systems*. DEC, 1984.

UNIFECAF. Materiais da disciplina Sistemas Operacionais (unidades, videoaulas e textos-base). Ambiente Virtual de Aprendizagem.

Repositório do projeto: https://github.com/kaiooo20/kernellab